

# Cours Oracle

## Master UDB



UNIVERSITE  
DAKAR-BOURGUIBA

Mr Koundoul  
[birane.koundoul@uadb.edu.sn](mailto:birane.koundoul@uadb.edu.sn)

# Gestion des utilisateurs

# Architecture d'oracle

Une base de données a besoin d'au moins d'un administrateur pour son administration.

L'administrateur a pour tâche :

- ☐ Installer le serveur Oracle
- ☐ Planifier et créer des bases de données
- ☐ Gérer les structures physiques
- ☐ Créer et gérer des utilisateurs
- ☐ Gérer la sécurité
- ☐ optimiser les performances de la base de données
- ☐ Planifier et exécuter les sauvegardes et restaurations

# Le système oracle

- ❑ On distingue deux composantes dans un système d'information : les données et les traitements.
  - Les données sont représentées par une base de données
  - et les traitements par un ensemble d'applications informatiques et par des procédures manuelles.
  
- ❑ Une base de données Oracle comprend deux types d'informations: le **dictionnaire de données** et les **données**.
  - Le **dictionnaire de données** est un ensemble de tables qui décrivent la base ;
  - les **données** sont de deux types : les données des utilisateurs (essentiellement les tables et les index) et les données nécessaires au fonctionnement d'Oracle (exemple les segments de rollback).

# Les utilisateurs

Les utilisateurs de la base de données interagissent avec elle via des utilitaires (**exemple**: SQL\*Plus, SQL Developer) ou des applications.

Les tâches typiques d'un utilisateur de base de données incluent des tâches telles que : l'**insertion**, la **modification** et la **suppression** des données auxquelles il a accès.

Le mode le plus couramment utilisé pour créer un utilisateur est :

**Create user nomUser identified by motDePasse;**

Ce mode permet à l'utilisateur de se connecter avec un mot de passe défini lors de la création d'utilisateur (authentification par la base de données).

Une autre authentification nommée authentification par système d'exploitation. Dans ce cas de figure on remplace le mot de passe par **EXTERNALLY**.

# Les utilisateurs

Comme dans tout système multi-utilisateur, l'utilisateur d'un SGBD doit être identifié avant de pouvoir utiliser des ressources.

L'accès aux informations et à la base de données doit être contrôlé à des fins de sécurité et de cohérence.

Un utilisateur (*user* ou *username*) est identifié au niveau de la base par son nom et peut se connecter puis accéder aux objets de la base sous réserve d'avoir reçu un certain nombre de privilèges.

**NB** : un schéma est une collection nommée ( du nom de l'utilisateur qui en est propriétaire) d'objets (tables, vues, séquences, index, procédures, ...)

# Création utilisateur

Utilisez l'instruction **CREATE USER** pour créer et configurer un utilisateur de base de données, qui est un compte par lequel nous pouvons nous connecter à la base de données, et pour établir les moyens par lesquels Oracle Database autorise l'accès de l'utilisateur.

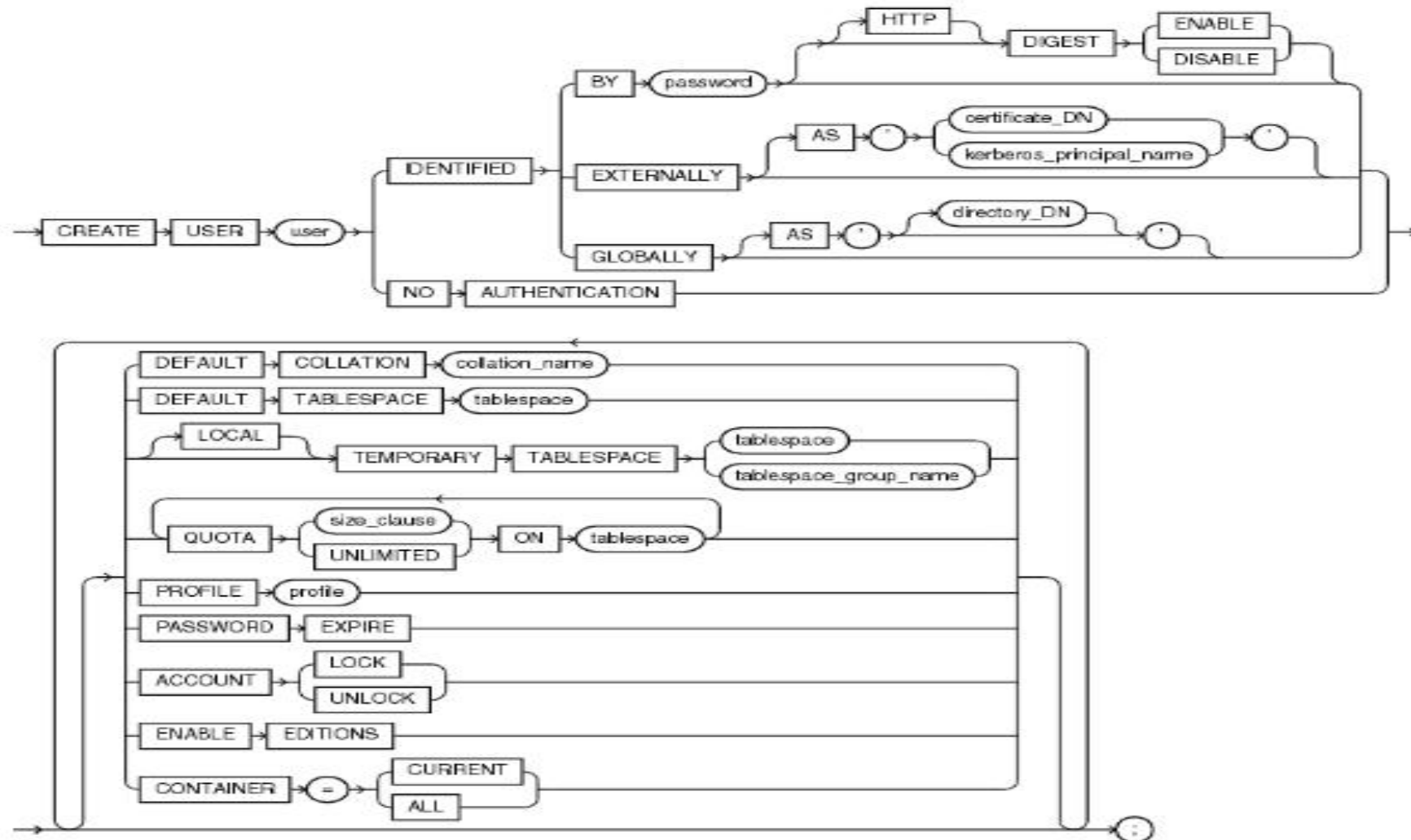
Nous devons disposer du privilège système **CREATE USER**. Lorsque nous créons un utilisateur à l'aide de l'instruction **CREATE USER**, le domaine de privilèges de l'utilisateur est vide. Pour se connecter à Oracle Database, un utilisateur doit disposer du privilège système **CREATE SESSION**. Par conséquent, après avoir créé un utilisateur, nous devons lui accorder au moins le privilège système **CREATE SESSION**.

# Création utilisateur

Pour spécifier la clause **CONTAINER**, nous devons être connecté à une base de données conteneur multitenant (CDB). Pour spécifier `CONTAINER = ALL`, le conteneur actuel doit être la racine. Pour spécifier `CONTAINER = CURRENT`, le conteneur actuel doit être une base de données enfichable (PDB).



# Création utilisateur



# Création utilisateur

Pour pouvoir créer un utilisateur, on doit posséder le privilège *CREATE USER*.

**CREATE USER** *utilisateur* **IDENTIFIED**

{ BY *motdePasse* | EXTERNALLY | GLOBALLY AS '*nomExterne*' }

[ DEFAULT TABLESPACE *nomTablespace*

[ QUOTA { entier [ K | M ] | UNLIMITED } ON *nomTablespace* ] ]

[ TEMPORARY TABLESPACE *nomTablespace*

[ QUOTA { entier [ K | M ] | UNLIMITED } ON *nomTablespace* ] ]

[ PROFILE *nomProfil* ] [ PASSWORD EXPIRE ] [ ACCOUNT { LOCK | UNLOCK } ] ;

# Création utilisateur

**IDENTIFIED** BY *motdePasse* permet d'affecter un mot de passe à un utilisateur local (cas le plus courant et le plus simple).

**IDENTIFIED** BY EXTERNALLY permet de se servir de l'authenticité du système d'exploitation pour s'identifier à Oracle.

**IDENTIFIED** BY GLOBALLY permet de se servir de l'authenticité d'un système d'annuaire.

**DEFAULT TABLESPACE** nomTablespace associe un espace disque de travail (appelé tablespace) à un utilisateur.

**TEMPORARY TABLESPACE** nomTablespace associe un espace disque temporaire (dans lequel certaines opérations se dérouleront) à l'utilisateur.

**QUOTA** permet de limiter ou pas chaque espace alloué.

**PROFILE** nomProfil affecte un profil (caractéristiques système relatives au CPU et aux connexions) à l'utilisateur.

# Création utilisateur

En l'absence de clause sur les espaces disque, le *tablespace* **SYSTEM** est associé à l'utilisateur en tant qu'espace de travail et d'espace temporaire. Il existe d'autres *tablespaces* créés par Oracle, on peut en citer **USERS** (celui qu'on devrait utiliser comme espace par défaut) et **TEMP** (celui qu'on devrait employer pour notre espace temporaire). On peut aussi créer nos propres espaces. La clause **ALTER USER** permet d'affecter un espace de travail ou temporaire différent de celui de départ.

En l'absence de profil, le profil **DEFAULT** est affecté à l'utilisateur.

Par défaut, les utilisateurs, une fois créés, n'ont aucun droit sur la base de données sur laquelle ils sont connectés.

# Création utilisateur

**User** : Indiquez le nom de l'utilisateur à créer. Ce nom ne peut contenir que des caractères du jeu de caractères de votre base de données et doit respecter les règles décrites dans la section "Règles de dénomination des objets de la base de données". Oracle recommande que le nom de l'utilisateur contienne au moins un caractère à un octet, que le jeu de caractères de la base de données contienne ou non des caractères à plusieurs octets.

Dans une base de données autre qu'une base de données, un nom d'utilisateur ne peut pas commencer par C## ou c##.

**La clause identified** vous permet d'indiquer comment Oracle Database authentifie l'utilisateur.

# Création utilisateur

## **BY password**

La clause BY password permet de créer un utilisateur local et indique que l'utilisateur doit spécifier un mot de passe pour se connecter à la base de données. Les mots de passe sont sensibles à la casse. Les mots de passe peuvent contenir n'importe quel caractère à un ou plusieurs octets, n'importe quel caractère spécial ou n'importe quelle combinaison de ces caractères du jeu de caractères de votre base de données, à l'exception du guillemet double (") et du caractère de retour. Si un mot de passe commence par un caractère non alphabétique ou contient un caractère autre qu'un caractère alphanumérique, le trait de soulignement (\_), le signe du dollar (\$) ou le signe dièse (#), il doit être placé entre guillemets doubles. Dans le cas contraire, la mise entre guillemets d'un mot de passe est facultative.

# Création utilisateur

## **[HTTP] DIGEST Clause**

Cette clause vous permet d'ACTIVER ou de DÉACTIVER l'authentification d'accès HTTP Digest pour l'utilisateur. La valeur par défaut est DISABLE. Le mot clé HTTP est facultatif et est fourni pour des raisons de clarté sémantique.

## **EXTERNALLY Clause**

Spécifiez EXTERNALLY pour créer un utilisateur externe. Cet utilisateur doit être authentifié par un service externe, tel qu'un système d'exploitation ou un service tiers. Dans ce cas, Oracle Database s'appuie sur l'authentification par le système d'exploitation ou le service tiers pour s'assurer qu'un utilisateur externe spécifique a accès à un utilisateur de base de données spécifique.

# Création utilisateur

## **GLOBALLY Clause**

La clause GLOBALLY permet de créer un utilisateur global. Cet utilisateur doit être autorisé par le service d'annuaire de l'entreprise (Oracle Internet Directory).

## **NO AUTHENTICATION Clause**

Utilisez la clause NO AUTHENTICATION pour créer un schéma qui n'a pas de mot de passe et auquel on ne peut pas se connecter. Cette clause est destinée aux comptes de schéma uniquement et réduit la maintenance en supprimant les mots de passe par défaut et toute obligation de rotation du mot de passe.



# Création utilisateur

## Password expire :

Cette clause permet à l'utilisateur de changer son mot de passe lors de sa première connexion.

```
create user C##koundoul identified by koundoul password expire;
```

## Account lock / unlock

La clause account avec les valeurs lock ou unlock, permet de bloquer le compte de l'utilisateur. Ce dernier ne pourra pas y accéder si la valeur est lock lors de la création. Par défaut c'est unlock. L'administrateur peut débloquent le compte de l'utilisateur avec la commande :

```
alter user nomUser account unlock;
```

# Création utilisateur

Oracle Multitenant permet à une base de données Oracle Database de fonctionner comme une base de données de conteneurs (**CDB**). Une base de données de conteneurs consolide plusieurs bases de données enfichables (**PDB**), une collection portable de schémas, d'objets de schéma et d'objets autres que de schéma.

Dans une CDB, les exigences relatives à un nom d'utilisateur sont les suivantes :

- ❑ Le nom d'un utilisateur commun doit commencer par des caractères qui correspondent, sans tenir compte de la casse, au préfixe spécifié par le paramètre d'initialisation `COMMON_USER_PREFIX`. Par défaut, le préfixe est `C##`.

# Création utilisateur

Pour créer un utilisateur dans oracle 21c, il faut **C##SuiviDuNom** :

```
create user C##user1UDB identified by user1UDB;
```

Pour avoir la liste des tous les utilisateurs

```
select * from all_users;
```

Pour modifier un utilisateur, on utilise la commande:

```
alter user nomUser identified by nouveauMotDePasse;
```

Pour supprimer un utilisateur

```
drop user C##user1UDB cascade;
```

# Création utilisateur

Avec la version 21c, pour créer un utilisateur, il faut la règle citée précédemment. Pour contourner cela, nous allons procéder ainsi:

Commençons par vérifier les l'instance:

`show pdbs;` ou `show con_name;`

Pour activer une instance `alter session set container = ORCLPDB;`

**NB :** ORCLPDB le nom de la connexion.

Ce qui permet de créer un compte sans **C##SuiviDuNom;**

`create user user2 identified by user2UDB;`

Mais il faut ouvrir la base de données : `startup mount;`

On peut aussi utiliser la commande ci-dessous pour avoir le nom du service :

`select * from global_name;`

# Modification utilisateur

Pour pouvoir modifier les caractéristiques d'un utilisateur (autres que celle du mot de passe), on doit posséder le privilège *ALTER USER*.

## **ALTER USER** *utilisateur* **IDENTIFIED**

{ BY *motdePasse* [REPLACE *ancienMotdePasse*] | EXTERNALLY | GLOBALLY AS '*nomExterne*' }

[ DEFAULT TABLESPACE *nomTablespace*

[QUOTA { entier [ K | M ] | UNLIMITED } ON *nomTablespace* ] ]

[ TEMPORARY TABLESPACE *nomTablespace*

[QUOTA { entier [ K | M ] | UNLIMITED } ON *nomTablespace* ] ]

[ PROFILE *nomProfil* ]

[ DEFAULT ROLE { rôle1 [, rôle2]... | ALL [EXCEPT rôle1 [, rôle2] ... ] | NONE }

[ PASSWORD EXPIRE ] [ ACCOUNT { LOCK | UNLOCK } ] ;

Chaque utilisateur peut changer son propre mot de passe à l'aide de cette instruction. Les autres changements seront opérationnels aux prochaines sessions de l'utilisateur mais pas à la session courante (cas de l'utilisateur qui déclare un espace de travail alors qu'il est couramment connecté à un autre).

PASSWORD EXPIRE oblige l'utilisateur à changer son mot de passe à la prochaine connexion.

DEFAULT ROLE affecte à l'utilisateur des rôles qui sont en fait des ensembles de privilèges.

# Suppression d'utilisateur

Pour pouvoir modifier les caractéristiques d'un utilisateur (autres que celle du mot de passe), on doit posséder le privilège **DROP USER**.

**DROP USER** *utilisateur* [ CASCADE] ;

Oracle ne supprime pas par défaut un utilisateur ayant des objets (tables, séquences, index, déclencheurs, etc.). L'option **CASCADE** force la suppression et détruit tous les objets du schéma de l'utilisateur.

Les contraintes d'intégrité d'autres schémas qui référençaient des tables du schéma à détruire sont aussi supprimées.

Les vues, synonymes, procédures ou fonctions cataloguées définies à partir du schéma détruit mais présents dans d'autres schémas ne sont pas supprimés mais invalidés.

Les rôles définis par l'utilisateur à supprimer ne sont pas détruits par l'instruction *DROP USER*.

Un utilisateur connecté ne peut pas être supprimé en direct avec cette commande. Pour forcer cette suppression, il faut arrêter ses sessions par la commande **ALTER SYSTEM** et l'option **KILL SESSION**.

Si on désire juste effacer l'utilisateur en tant qu'entrée dans la base sans supprimer ses objets, on utilisera **REVOKE** du privilège **CREATE SESSION**.

# Modification quota et tablespace utilisateur

Pour modifier le quota, on peut utiliser la commande :

```
alter user nomUser  
quota 30M on tablespace1  
quota 50M on tablespace;
```

Pour modifier les tablespaces par défaut:

```
alter user nomUser  
default tablespace nomTablespace  
temporary tablespace nomTablespace;
```

# Privilèges

Un privilège (sous-entendu utilisateur) est un droit d'exécuter une certaine instruction SQL (on parle de *privilège système*), ou un droit d'accéder à un certain objet d'un autre schéma (on parle de *privilège objet*).

Les privilèges système diffèrent sensiblement d'un SGBD à un autre. En revanche, les privilèges objets sont les mêmes et sont tous pris en charge via les instructions **GRANT** et **REVOKE**.

Les privilèges assortis de la mention ANY donne la possibilité au bénéficiaire de s'en servir dans tout schéma (n'incluant pas par défaut celui de l'utilisateur SYS).

Par exemple, le privilège CREATE ANY TABLE permet de créer des tables dans tout schéma alors que le privilège CREATE TABLE ne permet de créer des tables que dans son propre schéma.

## Quelques vues utilises:

**dba\_users** : des informations sur les utilisateurs

**dba\_ts\_quotas** : des informations sur les quotas



# Privilèges

Il existe une centaine de privilèges système. on peut citer, par exemple, la création d'utilisateurs (CREATE USER), la création et la suppression de tables (CREATE/DROP TABLE), la sauvegarde des tables (BACKUP ANY TABLE), etc.

| Privilège            | ALTER | CREATE | DROP | Autre                                         |
|----------------------|-------|--------|------|-----------------------------------------------|
| INDEX                |       | X      |      | QUERY REWRITE (index basés sur des fonctions) |
| ANY INDEX            | X     | X      | X    |                                               |
| TABLE                |       | X      |      |                                               |
| ANY TABLE            | X     | X      | X    | BACKUP, INSERT, DELETE, SELECT, UPDATE        |
| USER                 | X     | X      | X    | BECOME (pour des importations de bases)       |
| PROFILE              | X     | X      | X    |                                               |
| SEQUENCE             |       | X      |      |                                               |
| ANY SEQUENCE         | X     | X      | X    | SELECT (pour utiliser toute séquence)         |
| ANY OBJECT PRIVILEGE |       |        |      | Pour manipuler tout objet                     |
| Etc.                 |       |        |      |                                               |

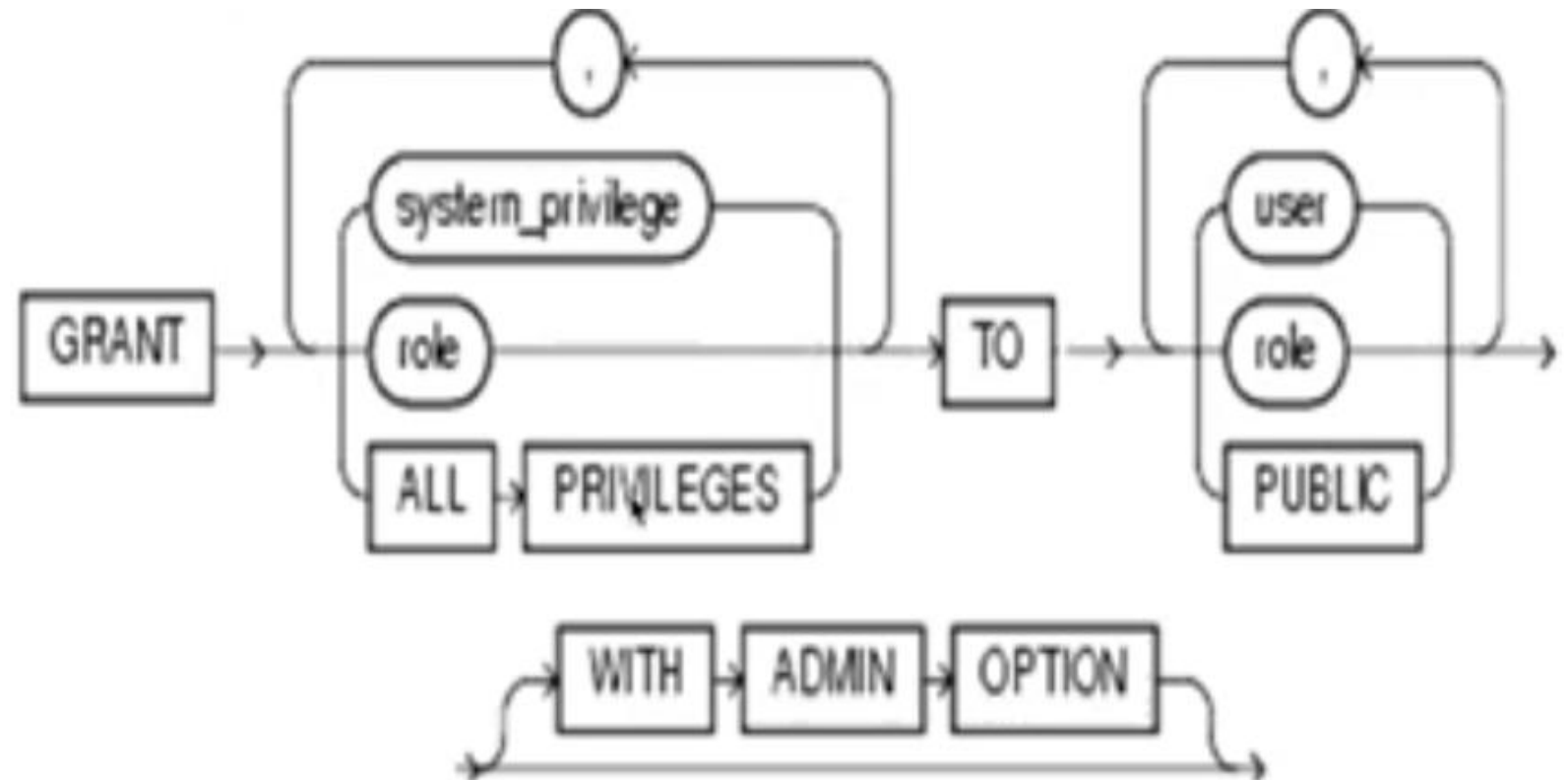
# Rôles et privilèges

Les rôles et privilèges sont définis pour sécuriser l'accès à la base de données.

Deux types de privilèges existent :

- ❑ privilèges système
- ❑ privilèges objet

**Pour les privilèges systèmes:**



# Rôles et privilèges

```
create user user2 identified by user2UDB;
```

Cette commande va générer une erreur car la règle n'est pas respectée.

```
grant create session to C##user1UDB;
```

Pour attribuer le privilège à l'utilisateur de se connecter. La création de l'utilisateur ne suffit pas.

## Rappel :

```
show pdbs; == pour voir l'état des instance =
```

Pour lancer la commande **show pdbs**; il faut se connecter en tant que dba;

```
connect system as sysdba;
```

# privilèges Systèmes

La commande GRANT permet d'attribuer un ou plusieurs privilèges à un ou plusieurs bénéficiaires.

**GRANT** { *privilègeSystème* | *nomRôle* | ALL PRIVILEGES }

[, { *privilègeSystème* | *nomRôle* | ALL PRIVILEGES }] ...

**TO** { *utilisateur* | *nomRôle* | PUBLIC }

[, { *utilisateur* | *nomRôle* | PUBLIC }] ...

[IDENTIFIED BY *motdePasse*] [WITH ADMIN OPTION] ;

*privilègeSystème* : description du privilège système (**exemple** : CREATE TABLE, CREATE SESSION, etc.).

- ❑ ALL PRIVILEGES : tous les privilèges système.
- ❑ PUBLIC : pour attribuer le(s) privilège(s) à tous les utilisateurs.
- ❑ IDENTIFIED BY : désigne un utilisateur encore inexistant dans la base. Cette option n'est pas valide si le bénéficiaire est un rôle ou est PUBLIC.
- ❑ WITH ADMIN OPTION : permet d'attribuer aux bénéficiaires le droit de retransmettre le(s) privilège(s) à une tierce personne (utilisateur(s) ou rôle(s)).

# privileges Systèmes

L'utilisateur qui exécute cette commande doit avoir reçu lui-même le droit de transmettre ces privilèges. Dans le cas des utilisateurs **SYS** et **SYSTEM**, la question ne se pose pas car ils ont tous les droits.

# Les privilèges systèmes

Après création de l'utilisateur, on peut lui attribuer des privilèges système pour que le (s) puisse (nt) faire un certains nombre de tâches. Plusieurs privilèges existent :

```
grant connect to user2UDB;  
grant connect, resource, dba to user2UDB;  
grant create session to user2UDB;  
grant unlimited tablespace to user2UDB;
```

```
grant alter any table to user2UDB;  
grant alter any procedure to user2UDB;  
grant alter any trigger to user2UDB;
```

```
grant create view, create procedure, create sequence,  
create trigger to user2UDB;
```

```
grant delete any table to user2UDB;  
grant drop any procedure to user2UDB;  
grant drop any trigger to user2UDB;  
grant any view to user2UDB;
```

La liste des privilèges à attribuer est exhaustive.

Note : éviter d'attribuer tous les privilèges à un utilisateur. C'est à dire : `grant all privileges to user2UDB;`

# Les privilèges systèmes

Attribuer des privilèges système à un utilisateur:

| Database       |                                     |
|----------------|-------------------------------------|
| Alter database | Modification de la base de données  |
| Alter system   | Autorise l'instruction Alter System |

| Session        |                                     |
|----------------|-------------------------------------|
| Create session | Connexion à la base de données      |
| Alter session  | Autorise l'instruction Alter System |

# Les privilèges systèmes

Attribuer des privilèges système à un utilisateur:

| Tables           |                                                         |
|------------------|---------------------------------------------------------|
| Create table     | Création de table dans le schéma                        |
| Create any table | Création de table dans n'importe quel schéma            |
| Alter any table  | Modification de table ou vue dans n'importe quel schéma |
| Drop any table   | Suppression de table dans n'importe quel schéma         |
| Select any table | Intéroger les tables de n'importe quel schéma           |

| Tablespace           |                                  |
|----------------------|----------------------------------|
| Create tablespace    | Création de tablespace           |
| Alter tablespace     | Modification de tablespace       |
| Drop tablespace      | Suppression de tablespace        |
| Unlimited tablespace | Quota illimité sur le tablespace |



# Révocation de privilèges système

La commande **REVOKE** permet de révoquer un ou plusieurs privilèges.

**REVOKE** { *privilègeSystème* | *nomRôle* | ALL PRIVILEGES }

[, { *privilègeSystème* | *nomRôle* }] ...

**FROM** { *utilisateur* | *nomRôle* | PUBLIC }

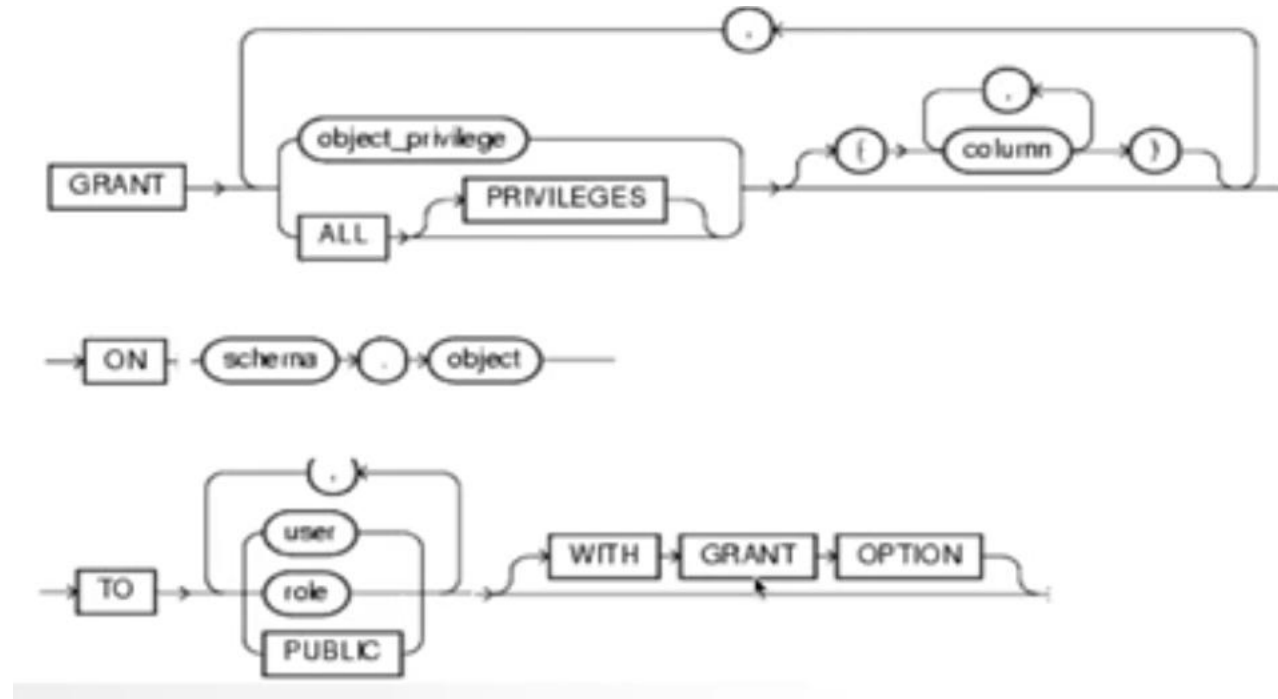
[, { *utilisateur* | *nomRôle* }] ... ;

Les options sont les mêmes que pour la commande **GRANT**.

- ❑ ALL PRIVILEGES : valable si l'utilisateur ou le rôle ont tous les privilèges système.
- ❑ PUBLIC : pour annuler le(s) privilège(s) à chaque utilisateur ayant reçu ce(s) privilège(s) par l'option PUBLIC.
- ❑ Pour pouvoir révoquer un privilège ou un rôle, on doit détenir au préalable ce privilège avec l'option WITH ADMIN OPTION.

# Les privilèges objets

Attribuer des privilèges objet à un utilisateur:



# Les privilèges objets

Ils sont relatifs aux données de la base et aux action sur les objets (table, vue, séquence, procédure).

Chaque type d'objet a différents privilèges associés.

| Privilège  | Table | Vue | Séquence | Programme PL/SQL |
|------------|-------|-----|----------|------------------|
| ALTER      | X     |     | X        | X                |
| DELETE     | X     | X   |          |                  |
| EXECUTE    |       |     |          | X                |
| INDEX      | X     |     |          |                  |
| INSERT     | X     | X   |          |                  |
| REFERENCES | X     |     |          |                  |
| SELECT     | X     | X   | X        |                  |
| UPDATE     | X     | X   |          |                  |
| ...        |       |     |          |                  |

# Les privilèges objets

La commande GRANT permet d'attribuer un (ou plusieurs) privilège sur un (ou plusieurs) objet à un (ou plusieurs) bénéficiaire.

```
GRANT { privilègeObjet | nomRôle | ALL PRIVILEGES } [ colonne1 [, colonne2] ... ]  
[, { privilègeObjet | nomRôle | ALL PRIVILEGES } [ colonne1 [, colonne2] ... ] ] ...  
ON { [schéma.]nomObjet | { DIRECTORY nomRepertoire  
| JAVA {SOURCE | RESOURCE } [schéma.]nomObjet } }  
TO { utilisateur | nomRôle | PUBLIC } [, { utilisateur | nomRôle | PUBLIC } ] ...  
[WITH ADMIN OPTION];
```

L'utilisateur qui exécute cette commande doit avoir reçu lui-même le droit de transmettre ces privilèges sauf s'il s'agit de ses propres objets pour lesquels il possède automatiquement les privilèges avec l'option **GRANT OPTION**.

L'option REFERENCES permet d'implanter une contrainte d'intégrité entre deux tables de schémas distincts.

*privilègeSystème* : description du privilège système (**exemple** : SELECT, DELETE, etc.).

- ❑ ALL PRIVILEGES : tous les privilèges système.
- ❑ PUBLIC : pour attribuer le(s) privilège(s) à tous les utilisateurs.
- ❑ WITH ADMIN OPTION : permet d'attribuer aux bénéficiaires le droit de retransmettre le(s) privilège(s) à une tierce personne (utilisateur(s) ou rôle(s)).

*colonne* : précise la ou les colonnes sur lesquelles se porte le privilège *INSERT*, *REFERENCES*, ou *UPDATE*.

# Les privilèges objets

Attribuer un utilisateur d'accéder à la table de l'utilisateur SYS:

grant

**select,**

**insert,**

**update,**

**delete**

on **SYS.nomTable**

to **nomUtilisateuraAttribuer;**

Attribuer un utilisateur d'accéder à des colonnes d'une table de l'utilisateur SYS:

grant

**update (colonne1, colonne2,colonneN)**

on **SYS.nomTable**

to **nomUtilisateuraAttribuer;**

# Les privilèges objets

| Tables     |                                           |
|------------|-------------------------------------------|
| alter      | Modification de la définition de la table |
| delete     | Supprimer des lignes de la table          |
| insert     | Insérer des ligne dans la table           |
| update     | Mise à jour de la table                   |
| select     | Intéroger la table                        |
| references | Création d'une contrainte d'intégrité     |

# Révocation de privilèges objets

La commande **REVOKE** permet de révoquer un ou plusieurs privilèges objets.

```
REVOKE { privilègeObjet | ALL PRIVILEGES } [ colonne1 [, colonne2] ... ]  
[, { privilègeObjet | ALL PRIVILEGES } [ colonne1 [, colonne2] ... ] ] ...  
ON { [schéma.]nomObjet | { DIRECTORY nomRepertoire  
| JAVA {SOURCE | RESOURCE } [schéma.]nomObjet } }  
FROM { utilisateur | nomRôle | PUBLIC } [, { utilisateur | nomRôle | PUBLIC } ] ...  
[CASCADE CONSTRAINTS] [FORCE] ;
```

Pour pouvoir révoquer un privilège objet, on doit détenir au cette permission ou le privilège système ANY OBJECT PRIVILEGE.

Il n'est pas possible d'annuler un privilège objet accordé avec l'option WITH ADMIN OPTION.

Certaines options sont similaires à celles de la commande **GRANT**.

- ❑ **CASCADE CONSTRAINTS** : concerne les privilèges REFERENCES ou ALL PRIVILEGES. Cette option permet de supprimer la contrainte référentielle entre deux tables de schémas distincts.
- ❑ **FORCE** : concerne les privilèges EXECUTE sur les types (extensions **SQL3**). En ce cas, tous les objets dépendants (types, tables ou vues) sont marqués INVALID et les index sont notés UNISABLE.

# informations utilisateur

Pour se connecter en tant qu'administrateur

nom user admin = sys

mot de passe : lors de l'installation.

nom service : ORCLPDB

port : 1521

nom machine = vérifie le listener C:\Users\Oracle21c\homes\OraDB21Home1\network\admin\listener

rôle : sysdba



# informations utilisateur

Pour se connecter avec le nouveau utilisateur crée sans C##, il faut taper cette commande

`show con_name;` pour connaitre le nom de la connexion

- **ORCLPDB**

`sqlplus user2UDB/user2UDB@127.0.0.1:1521/ORCLPDB`

- user2UDB= nom user
- user2UDB = mot de passe
- 127.0.0.1 adresse machine : se trouve

C:\Users\Oracle21c\homes\OraDB21Home1\network\admin\listener

- ORCLPDB = nom connexion

# Connexion utilisateur

Pour vous connecter à la base de données de conteneur, utilisez :

```
sqlplus sys/[mot de passe]@//localhost:1521/ORCL as sysdba
```

Pour vous connecter à la première base de données enfichable, utilisez :

```
sqlplus sys/[mot de passe]@//localhost:1521/ORCLPDB as sysdba
```

Oracle Multitenant permet à une base de données Oracle Database de fonctionner comme une base de données de conteneurs (CDB). Une base de données de conteneurs consolide plusieurs bases de données enfichables (PDB), une collection portable de schémas, d'objets de schéma et d'objets autres que de schéma.

# Connexion utilisateur

Pour régler le problème de SQL Developer :

aller sur le console et taper :

1) sqlplus

2) / as sysdba

3) select sys\_context('userenv','instance\_name') from DUAL;

show con\_name; = CDB\$ROOT

# Connexion utilisateur

## Exercice

1. Connectez en tant qu'utilisateur SYS. Créez deux utilisateurs (Un utilisateur udb1 dans le conteneur CDB\$ROOT et un utilisateur udb2 dans le conteneur ORCLPDB).
2. Créer la table client (idClient, prenomn, nom, telephone) avec idClient auto incrémental. Cette table est créée par l'utilisateur udb1.
3. Créer deux tables **produit** (idProduit, nomProd, quantite, prix, #idCat) et la table **categorie**(idCat, nomCat, description) appartenant à udb2.
4. Remplissez les trois tables.
5. Connecter en tant que SYS et attribue le privilège aux utilisateurs le privilège de créer les tables, de sequence;
6. en tant que SYS, créer un troisième utilisateur nommé udb3.
7. Connecte en tant udb2 pour attribuer le privilège de sélection dans la table catégorie à udb2;
8. User udb3, lister les informations de la table catégorie.
9. User udb2, retirer le privilège de sélection à udb3.